algorithm has explored some execution sequence in $[E'_p.p.w'_p.w'']_{\simeq} = [E.w]_{\simeq}$, which contradicts the main assumption about w. □

For $p \in \mathit{final_sleep}(E)$, also define:

- w_p, as the longest prefix of w such that $E \vdash p \diamond w_p$,
- e_p, as the first event in $\mathit{dom}_{[E]}(w)$ which is not in w_p. Such an event e_p must exist, otherwise $w_p = w$, which implies $E \vdash p \diamond w$, which implies $p \in \mathit{WI}_{[E]}(w)$, which contradicts Claim 7.5.

Finally, define:

- $q \in \mathit{final_sleep}(E)$, such that w_q is a longest prefix among w_p. If there are several processes $p \in \mathit{final_sleep}(E)$ such that w_p is the same longest prefix, then pick q such that $E'_q.q$ is minimal (w.r.t. $\propto$).
- w_R, as $w_q.\widehat{e_q}$.
- Sleep' as the initial sleep set argument in the exploration of $E'_q.q$, i.e., it started by calling $\mathit{Explore}(E'_q.q, \mathit{Sleep}', \cdot)$.

Claim 7.6. $E'_q \vdash q.w'_q.w_R$.

Proof. Since $E'_q \vdash q$ (actually explored) and $E'_q \vdash q \diamond (w'_q.w_q)$, it follows that $E'_q \vdash w'_q.w_q.q$. Given also that $E'_q \vdash w'_q.w_q.e_q$ and q does not disable e_q (since q and $\widehat{e_q}$ are different), we conclude that $E'_q \vdash w'_q.w_q.q.e_q$ and hence $E'_q \vdash q.w'_q.w_q.e_q$ which can be written as $E'_q \vdash q.w'_q.w_R$. □

Claim 7.7. $\mathit{WI}_{[E'_q.q]}(w'_q.w_R) \cap \mathit{Sleep}' = \emptyset$.

Proof. We will establish the stronger property $\mathit{WI}_{[E'_q.q]}(w'_q.w_q) \cap \mathit{Sleep}' = \emptyset$. Claim 7.6 has shown that the execution sequence is valid. The proof is then by contradiction: Assume that some process p is in $\mathit{WI}_{[E'_q.q]}(w'_q.w_q) \cap \mathit{Sleep}'$.

By the construction of Sleep' at line 17, the process p must be in $\mathit{sleep}(E'_q)$ just before the call to $\mathit{Explore}(E'_q.q, \mathit{Sleep}', \cdot)$ and satisfy $E'_q \vdash p \diamond q$. From $p \in \mathit{WI}_{[E'_q.q]}(w'_q.w_q)$, and $E'_q \vdash p \diamond q$, and Property 4.7(5), it follows that $p \in \mathit{WI}_{[E'_q]}(q.w'_q.w_q)$, which, using $E'_q.q.w'_q.w_q \simeq E'_q.w'_q.w_q.q$ (which follows from $E'_q \vdash q \diamond (w'_q.w_q)$), implies $p \in \mathit{WI}_{[E'_q]}(w'_q.w_q.q)$, which by Property 4.7(4) implies $p \in \mathit{WI}_{[E'_q]}(w'_q.w_q)$.

Since, again by Property 4.7(4), $p \in \mathit{WI}_{[E'_q]}(w'_q.w_q)$ also implies $p \in \mathit{WI}_{[E'_q]}(w'_q)$, during exploration of $E'_q.w'_q$ no event in w'_q removes p from the sleep set. Thus, $p \notin w'_q$ and p will end up in $\mathit{sleep}(E'_q.w'_q)$ and from there in $\mathit{final_sleep}(E'_q.w'_q)$, which means $p \in \mathit{final_sleep}(E)$. It hence cannot hold that $p \in I_{[E]}(w_q)$, since this by Property 4.2(1) implies $p \in I_{[E]}(w)$, hence $p \in \mathit{WI}_{[E]}(w)$, violating Claim 7.5. Since there can be no event in w_q happening-before p (otherwise the same event would prohibit $p \in \mathit{WI}_{[E'_q]}(w'_q.w_q)$) the only way for $p \notin I_{[E]}(w_q)$ is by $p \notin w_q$.

Therefore $p \notin w'_q.w_q$ which by $p \in \mathit{WI}_{[E'_q]}(w'_q.w_q)$ entails $E'_q \vdash p \diamond w'_q.w_q$. By choice of q, we then have necessarily that $e_p = e_q$ (otherwise w_p would be longer than w_q). But since among the processes p with $e_p = e_q$ we chose q to be the first one for which a call of the form $\mathit{Explore}(E'_q.p, \cdot, \cdot)$ was performed, we have that $p \notin \mathit{sleep}(E'_q)$ just before the call to $\mathit{Explore}(E'_q.q, \mathit{Sleep}', \cdot)$, whence $p \notin \mathit{Sleep}'$. Thus we have a contradiction. □

Claim 7.8. *Let z' be any sequence such that $E'_q.q.w'_q.w_R.z'$ is maximal (such a z' can always be found, since $E'_q.q.w'_q.w_R$ is an execution sequence). Then, the algorithm explores some sequence $E'_q.q.z$ in $[E'_q.q.w'_q.w_R.z']_{\simeq}$.*

Proof. From Claim 7.7, it follows $\mathit{WI}_{[E'_q.q]}(w'_q.w_R.z') \cap \mathit{Sleep}' = \emptyset$. Therefore, no execution sequence in $[E'_q.q.w'_q.w_R.z']_{\simeq}$ was explored before the call to $\mathit{Explore}(E'_q.q, \mathit{Sleep}', \cdot)$, otherwise,